Linear Regression

1. Introduction to Linear Regression

1.1 What is Linear Regression?

Linear Regression is a supervised machine learning algorithm used to predict a **continuous numerical value** by learning the relationship between input variables and an output variable.

Examples:

- Predicting salary based on years of experience.

Experience in Years	0	2	4	5	6
Salary	2,00,000	4,00,000	8,00,000	10,00,000	12,00,000

What would be the **salary** of a person with **3 years of Experience?**

~ ₹ 650000 per Year



- Predicting house prices based on size.
- Predicting electricity consumption based on temperature.
- Predicting a student's marks based on study hours.

1.2 Simple Linear Regression

Simple Linear Regression uses **one independent variable** to predict one dependent variable.

For example, to predict salary based on years of experience:

- Independent variable (x): Years of experience
- Dependent variable (y): Salary

The model attempts to fit a straight line that captures the overall trend.

$$\hat{y} = wx + b$$

Where:

- x = input feature
- $\hat{y}$ = predicted output
- w = weight or slope
- b = bias or intercept

Slope w: Indicates how much the predicted output changes when the input increases by one unit. If w = 5000, the model predicts salary increases by 5,000 units for each additional year of experience.

Intercept b: The predicted output when x = 0. If b = 20,000, the model predicts a salary of 20,000 units at zero years of experience.

1.3 Multiple Linear Regression

Multiple Linear Regression uses **two or more independent variables** to predict an output. For example, salary may depend on experience, educational qualification, and relevant skills.

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Each w_i represents the weight associated with a feature. With one input feature, the model represents a line in two-dimensional space. With two input features, it can represent a plane in three-dimensional space. With more features, it represents a hyperplane in higher-dimensional space.

1.4 Advantages and Disadvantages

Advantages	Disadvantages
Simple to understand and implement	Cannot naturally represent complex nonlinear relationships
Computationally efficient for many datasets	Sensitive to outliers, especially with squared-error loss
Easy to interpret the effect of features in the fitted model	May underfit when the actual relationship is nonlinear
Useful as a baseline for regression problems	Performance depends on feature and data quality

A straight line cannot adequately capture a sine-wave relationship without transforming features or using a more suitable model.

2. Mathematical Foundation of Linear Regression

2.1 The Equation of a Line

The equation of a straight line is y = mx + c. In machine learning, we commonly write:

$$\hat{y} = wx + b$$

Symbol	Meaning
x	Input feature
y	Actual target value
$\hat{y}$	Predicted target value
w	Weight or slope

Symbol	Meaning
b	Bias or intercept

2.2 What Is a Best-Fit Line?

A dataset may have many possible straight lines, but not all represent the data equally well.

Experience (x)	Actual Salary (y)
1	25,000
2	30,000
3	35,000
4	40,000
5	45,000

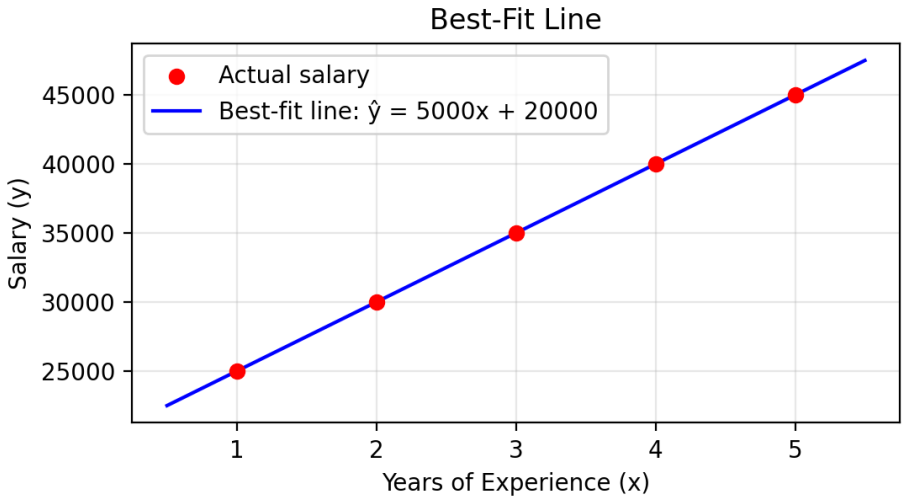
A line that captures this relationship is:

$$\hat{y} = 5000x + 20000$$

For x = 3:

$$\hat{y} = 5000(3) + 20000 = 35000$$

The prediction matches the actual salary for this example. Real-world data usually contains noise, so predictions do not match every observation perfectly. Linear Regression seeks a line that minimizes an overall measure of prediction error. This is the idea behind the **best-fit line**.



2.3 Actual Values, Predictions, and Errors

For each training example:

$$\hat{y}_i = wx_i + b$$

The prediction error, also called the residual, is:

$$e_i = y_i - \hat{y}_i$$

For example, if actual salary is 40,000 and predicted salary is 37,000:

$$e = 40000 - 37000 = 3000$$

Some residuals can be positive and others negative. Simply adding them could allow positive and negative errors to cancel out, so a loss function provides a more useful measure of overall error.

2.4 Loss Function and Mean Squared Error

A **loss function** measures the difference between actual and predicted values. Training aims to find model parameters that minimize the chosen loss. Lower the loss higher the accuracy.

Mean Squared Error (MSE) is a common loss for Linear Regression:

$$MSE = (1/n)\ \Sigma(i=1 \text{ to } n)\ (y_i - \hat{y}_i)^2$$

Where n is the number of examples, y_i is the actual value, and $\hat{y}_i$ is the prediction.

Steps:

1. Calculate predictions.
2. Subtract each prediction from its actual value.
3. Square each error.
4. Calculate the average of the squared errors.

Example

Actual value	Predicted value	Error	Squared error
3	2	1	1
5	7	-2	4

$$MSE = (1 + 4)/2 = 2.5$$

Why square the errors?

- Squaring prevents positive and negative errors from cancelling out.
- Larger errors receive a greater penalty.
- The function is convenient for gradient-based optimization.

Squared error is sensitive to outliers because large errors contribute disproportionately.

2.5 How Does the Model Learn?

Linear Regression adjusts its parameters, w and b, to reduce the loss:

1. Initialize the parameters.
2. Calculate predictions using $\hat{y} = wx + b$.
3. Calculate the loss using MSE.
4. Calculate how the loss changes when w and b change.
5. Update the parameters to reduce the loss.
6. Repeat until a stopping condition is reached.

This iterative process is commonly performed using **Gradient Descent**.

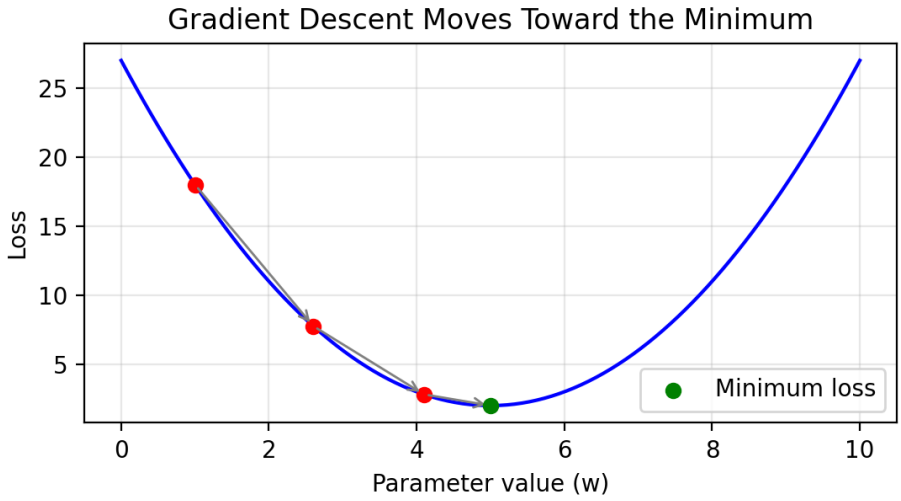
3. Gradient Descent

3.1 What Is Gradient Descent?

Gradient Descent is an optimization algorithm that updates model parameters in an attempt to minimize a loss function. For Linear Regression, it adjusts the weight w and bias b to minimize MSE.

3.2 Understanding the Loss Curve

For simple Linear Regression with MSE, the loss is a convex quadratic function of w and b. When one parameter is varied while the other is held fixed, the resulting curve is U-shaped when there is nonzero curvature in that direction.



- The horizontal axis represents a parameter value.
- The vertical axis represents loss.
- The minimum represents the lowest loss along that curve.
- Gradient Descent adjusts parameters to move toward lower loss.

For ordinary least-squares Linear Regression, MSE is convex and a global minimum exists. Gradient Descent can approach a global minimum with suitable settings, but it is not guaranteed to converge with every learning rate or initialization.

3.3 The Gradient Descent Update Rule

$$\theta_{new} = \theta_{old} - \alpha \, \partial L/\partial \theta$$

Where θ is a model parameter, L is the loss, α is the learning rate, and $\partial L/\partial \theta$ is the partial derivative of loss with respect to the parameter.

For Linear Regression:

$$w_{new} = w_{old} - \alpha \, \partial L/\partial w$$

$$b_{new} = b_{old} - \alpha \, \partial L/\partial b$$

The gradient points in the direction of steepest local increase, so subtracting it moves the parameter in the direction of steepest local decrease.

3.4 Deriving the Gradients for MSE

The prediction equation is:

$$\hat{y}_i = wx_i + b$$

The loss is:

$$L(w,b) = (1/n) \, \Sigma(i=1 \text{ to } n) \, (y_i - \hat{y}_i)^2$$

Substituting the prediction equation:

$$L(w,b) = (1/n) \, \Sigma(i=1 \text{ to } n) \, (y_i - wx_i - b)^2$$

Gradient with respect to weight w:

$$\partial L/\partial w = (1/n) \, \Sigma(i=1 \text{ to } n) \, 2(y_i - wx_i - b)(-x_i)$$

Therefore:

$$\partial L/\partial w = (2/n) \, \Sigma(i=1 \text{ to } n) \, (\hat{y}_i - y_i)x_i$$

Gradient with respect to bias b:

$$\partial L/\partial b = (1/n) \, \Sigma(i=1 \text{ to } n) \, 2(y_i - wx_i - b)(-1)$$

Therefore:

$$\partial L/\partial b = (2/n) \, \Sigma(i=1 \text{ to } n) \, (\hat{y}_i - y_i)$$

The update equations become:

$$w \leftarrow w - \alpha \, (2/n) \, \Sigma(i=1 \text{ to } n) \, (\hat{y}_i - y_i)x_i$$

$$b \leftarrow b - \alpha \, (2/n) \, \Sigma(i=1 \text{ to } n) \, (\hat{y}_i - y_i)$$

These formulas use the full training dataset for each update, so this is **Batch Gradient Descent**.

3.5 Understanding the Learning Rate

The learning rate α determines the size of each parameter update.

Learning rate	Possible effect
Too small	Loss may decrease very slowly
Suitable	Loss may decrease steadily toward a minimum
Too large	Updates may overshoot, oscillate, or diverge

The learning rate is a **hyperparameter**, chosen before training or tuned using validation data.

3.6 Iteration and Epoch

An **iteration** is one parameter-update step. For Batch Gradient Descent:

1. Calculate predictions on the training dataset.
2. Calculate gradients using the current parameters.
3. Update w and b.

An **epoch** is one complete pass through the training dataset. With Batch Gradient Descent, one epoch normally corresponds to one update. With mini-batch methods, one epoch contains multiple updates.

3.7 Convergence

Convergence occurs when the optimization process approaches a stable solution, such as a minimum of the loss function.

Training may stop when:

- A specified number of iterations has been completed.
- Loss changes very little between iterations.
- The gradient becomes sufficiently small.
- A validation-based stopping rule is reached.

Low training loss does not automatically guarantee good performance on unseen data. Evaluate the model on separate data.

4. Implementing Linear Regression from Scratch Using NumPy

The goal is to build a reusable `LinearRegression` class without using a prebuilt Linear Regression model from scikit-learn. NumPy handles the numerical calculations.

4.1 Structure of the Class

Component	Purpose
`__init__()`	Stores hyperparameters such as learning rate and number of iterations
`fit()`	Trains the model using training data
`predict()`	Generates predictions using learned parameters
`update_weights()`	Updates parameters using the gradients

4.2 Complete Code

```
import numpy as np



class LinearRegression:
    def __init__(self, learning_rate=0.01, n_iters=1000):
        # Hyperparameters
        self.learning_rate = learning_rate
        self.n_iters = n_iters

        # Parameters learned during training
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)

        # This implementation expects one feature
        if X.ndim == 1:
            X = X.reshape(-1, 1)

        if X.ndim != 2 or X.shape[1] != 1:
            raise ValueError("X must contain exactly one feature.")

        if y.ndim != 1:
            raise ValueError("y must be a one-dimensional array.")

        if X.shape[0] != y.shape[0]:
            raise ValueError("X and y must contain the same number of samples.")

        if X.shape[0] == 0:
            raise ValueError("Training data cannot be empty.")

        n_samples = X.shape[0]

        # Initialize parameters
        self.weights = 0.0
```

```
        self.bias = 0.0

        # Gradient Descent
        for _ in range(self.n_iters):
            y_predicted = self.predict(X)

            # Gradients of MSE
            dw = (2 / n_samples) * np.sum(
                X[:, 0] * (y_predicted - y)
            )
            db = (2 / n_samples) * np.sum(
                y_predicted - y
            )

            self.update_weights(dw, db)

    def update_weights(self, dw, db):
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db

    def predict(self, X):
        X = np.asarray(X, dtype=float)

        if X.ndim == 1:
            X = X.reshape(-1, 1)

        if X.ndim != 2 or X.shape[1] != 1:
            raise ValueError("X must contain exactly one feature.")

        if self.weights is None or self.bias is None:
            raise ValueError("Train the model using fit() first.")

        return self.weights * X[:, 0] + self.bias
```

4.3 Understanding the Important Lines

Initializing hyperparameters

```
def __init__(self, learning_rate=0.01, n_iters=1000):
    self.learning_rate = learning_rate
    self.n_iters = n_iters
```

- `learning_rate` controls the size of each update.
- `n_iters` determines how many gradient updates to perform.

Initializing parameters

```
self.weights = 0.0
self.bias = 0.0
```

The model starts with a weight and bias of zero. Training changes these values.

Counting examples

```
n_samples = X.shape[0]
```

`X.shape[0]` gives the number of rows, which corresponds to the number of training examples.

Calculating predictions

```
y_predicted = self.predict(X)
```

The current weight and bias calculate a prediction for every training example.

Calculating gradients

```
dw = (2 / n_samples) * np.sum(
    X[:, 0] * (y_predicted - y)
)
db = (2 / n_samples) * np.sum(
    y_predicted - y
)
```

These implement:

$$\partial L/\partial w = (2/n)\ \Sigma(i=1 \text{ to } n)\ (\hat{y}_i - y_i)x_i$$

$$\partial L/\partial b = (2/n)\ \Sigma(i=1 \text{ to } n)\ (\hat{y}_i - y_i)$$

`X[:, 0]` selects the single feature for every sample, and `np.sum()` adds the values.

Updating parameters

```
self.weights -= self.learning_rate * dw
self.bias -= self.learning_rate * db
```

This applies the Gradient Descent update rule.

Making predictions

```
return self.weights * X[:, 0] + self.bias
```

After training, prediction is simply $\hat{y} = wx + b$.

4.4 Test the Model on a Small Dataset

Run this after the class definition:

```
import numpy as np

X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([25, 30, 35, 40, 45], dtype=float)

model = LinearRegression(learning_rate=0.01, n_iters=2000)
model.fit(X, y)

new_experience = np.array([6, 7], dtype=float)
predictions = model.predict(new_experience)

print("Learned weight:", model.weights)
print("Learned bias:", model.bias)
print("Predictions:", predictions)
```

The data follows the exact relationship:

$$y = 5x + 20$$

Therefore, expected predictions for x = 6 and x = 7 are 50 and 55, respectively. Gradient Descent approaches the solution iteratively, so printed values may be close to, rather than exactly equal to, these theoretical values.

5. Applying the Model to a Salary Dataset

This workflow applies the custom implementation to a dataset containing years of experience and salary.

1. Load the dataset using Pandas.
2. Inspect the data and check for missing values.
3. Separate the input feature and target.
4. Split the data into training and test sets.
5. Train the custom model.
6. Predict salaries for the test set.
7. Evaluate and visualize predictions.
8. Save the class in a separate Python file for reuse.

5.1 Load and Inspect the Dataset

Assume the CSV file is named `Salary_Data.csv` and contains columns `YearsExperience` and `Salary`.

```
import pandas as pd

df = pd.read_csv("Salary_Data.csv")

print(df.head())
print(df.info())
print(df.isnull().sum())
```

- `pd.read_csv()` loads the CSV into a DataFrame.
- `df.head()` displays the first five rows.
- `df.info()` shows column names, data types, and non-null counts.
- `df.isnull().sum()` counts missing values in each column.

Before training, confirm that the required columns exist, values are numeric, and missing values have been handled appropriately.

5.2 Prepare Features and Target

```
X = df[["YearsExperience"]].to_numpy()
y = df["Salary"].to_numpy()
```

The double brackets preserve `X` as a two-dimensional feature matrix:

$$X \in \mathbb{R}^{n \times 1}$$

The target `y` is a one-dimensional array.

5.3 Split into Training and Test Sets

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

- `test_size=0.2` reserves 20% of examples for testing.
- `random_state=42` makes the random split reproducible.
- `X_train` and `y_train` fit the model.
- `X_test` and `y_test` evaluate it.

The test set should not be used to calculate gradients or fit the model parameters.

5.4 Train the Custom Model

```
model = LinearRegression(
    learning_rate=0.02,
    n_iters=1000
)

model.fit(X_train, y_train)
```

The model adjusts its weight and bias through Gradient Descent. The values `0.02` and `1000` are example hyperparameter settings, not universally optimal values. If updates are unstable or convergence is slow, the learning rate and preprocessing may need adjustment.

5.5 Make Predictions

```
y_pred = model.predict(X_test)
```

The model predicts salaries for held-out test examples.

5.6 Evaluate the Model

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print("MSE:", mse)
print("MAE:", mae)
```

- **MSE** measures average squared error.
- **MAE** measures average absolute error.

Lower values generally indicate better predictions when comparing models on the same target and evaluation dataset. MSE has squared target units, while MAE has the same units as salary.

5.7 Visualize Actual and Predicted Values

```
import matplotlib.pyplot as plt
import numpy as np

# Sort by experience so the predicted line is drawn in order
sort_idx = np.argsort(X_test[:, 0])
```

```
X_test_sorted = X_test[sort_idx]
y_pred_sorted = y_pred[sort_idx]

plt.scatter(
    X_test[:, 0],
    y_test,
    color="red",
    label="Actual salary"
)

plt.plot(
    X_test_sorted[:, 0],
    y_pred_sorted,
    color="blue",
    label="Predicted salary"
)

plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Linear Regression: Actual vs Predicted Salary")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

The red points represent actual test salaries, while the blue line represents model predictions across the test inputs. Sorting the inputs makes the predicted line easier to interpret. Consider numerical metrics as well as the visualization.

5.8 Save the Class in a Separate Python File

Save the class definition in a file named `linear_regression.py`.

Then, in another Python file or notebook in the same directory, import it:

```
from linear_regression import LinearRegression

model = LinearRegression(learning_rate=0.02, n_iters=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

This is called **modularization**. It keeps reusable model code separate from data preparation and evaluation.

6. Important Concepts to Remember

Concept	Meaning
Simple Linear Regression	Predicts an output using one input feature
Multiple Linear Regression	Predicts an output using multiple input features
Weight w	Controls the slope or feature contribution
Bias b	Intercept of the model
Prediction $\hat{y}$	Output estimated by the model
Residual	Difference between actual and predicted values
Loss function	Quantifies prediction error
MSE	Average of squared residuals
Gradient	Indicates how loss changes with a parameter
Gradient Descent	Iteratively updates parameters to reduce loss
Learning rate	Controls the size of each update
Iteration	One parameter-update step
Epoch	One complete pass through the training dataset
Convergence	The optimization process approaches a stable solution
`fit()`	Learns parameters from training data
`predict()`	Generates predictions using learned parameters
Training set	Data used to fit the model
Test set	Held-out data used for final evaluation
Modularization	Separating reusable code into its own module

7. Common Questions and Answers

Q1. What is the difference between simple and multiple Linear Regression?

Simple Linear Regression uses one input feature, while Multiple Linear Regression uses two or more input features.

Q2. Why do we need a loss function?

It measures the difference between actual and predicted values, giving the training algorithm an objective to minimize.

Q3. Why is MSE commonly used in Linear Regression?

MSE squares errors, prevents positive and negative residuals from cancelling, and penalizes larger errors more heavily. It also provides a differentiable objective that is convenient for optimization.

Q4. What is the purpose of Gradient Descent?

It iteratively updates model parameters in a direction intended to reduce the loss.

Q5. What happens if the learning rate is too high?

Parameter updates may overshoot the minimum, causing oscillation or divergence.

Q6. What is the difference between a parameter and a hyperparameter?

Weights and bias are model parameters learned during training. Learning rate and number of iterations are hyperparameters chosen to control training.

Q7. What is the purpose of `fit()`?

It learns the model's parameters from training data.

Q8. What is the purpose of `predict()`?

It uses learned parameters to generate predictions for input data.

Q9. Why split data into training and test sets?

The training set fits the model, while a separate test set estimates performance on unseen examples.

Q10. Why put the class in a separate `.py` file?

It makes the implementation reusable and keeps model code separate from data preparation and evaluation.

8. Quick Revision

The complete learning pipeline is:

1. **Training data:** Use input features X and actual targets y.
2. **Initialize parameters:** Choose initial weight w and bias b.
3. **Make predictions:** Calculate $\hat{y} = wx + b$.
4. **Calculate loss:** Measure prediction errors using MSE.
5. **Calculate gradients:** Determine how loss changes with w and b.
6. **Update parameters:** Apply Gradient Descent.
7. **Repeat:** Continue until the stopping condition is reached.
8. **Evaluate:** Use held-out test data to assess performance.
9. **Reuse:** Save the class in a separate Python module when needed.